

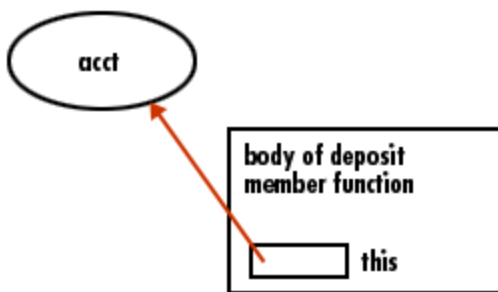
Topic 3.4

3.4: Accessing the Invoking Object

When a member function is invoked on an object (later, we will see static member functions which are not invoked on a specific object), the invoked function must be able to access the object on which it was invoked. This object is often referred to as the *invoking object*.

To achieve this, whenever an object invokes a member function, the compiler automatically passes an implicit extra parameter to the member function which is a pointer to the invoking object. *This implicit extra parameter* is a pointer to the invoking object.

Within the member function, the pointer to the invoking object is stored in a special variable known as *this*.



Within a member function, when we need to access members of the invoked object, we can use the *this* pointer:

```
void Account::deposit( Money amount )
{
    this->verify_initialization();
    this->account_balance += amount;
}
```

Within a member function, if we use unqualified member names of the class of which the function is a member, the compiler will automatically qualify those names with *this->*, so that the members of the invoking object are accessed:

```
void Account::deposit( Money amount )
{
    verify_initialization();
    account_balance += amount;
}
```

The word **this** is a new reserved word in C++. It is legal to use **this** only within member functions. You may assign **this** a value, making it point to a different object than the one on which the member function was invoked. Be extremely careful if you do this.

Topic 3.5 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-3-dot-5>)